assigning values to variables, or performing arithmetic operations or comparisons. In our first project, anything in between the curly braces { } is a statement; take the following, for example:

```
digitalWrite(9, HIGH);
```

This line of code is a statement that calls one of the Arduino library's built-in functions named `digitalWrite()`, which is used to turn on or off one of the Arduino digital pins previously configured as an output. We pass this function two pieces of information that it needs to do its job: the number of the pin and the state that we want the pin to be in. These additional parameters, specific to each function, are called **arguments** and are found inside the left and right parenthesis, with multiple arguments separated by **commas**. In this example, we are telling the Arduino environment to turn on digital pin 9 of the Arduino interface board, as explained later.

There are two kinds of statements used in programming C: simple statements, including the most recent example, as well as compound statements. **Simple statements** include function calls, such as our earlier call to `digitalWrite()`, assignment statements, and arithmetic operations and are always terminated with a semicolon. **Semicolons** (`;`) are used to separate one simple statement from another so that the development environment can tell them apart. Forgetting to end a simple statement with a semicolon will result in one of the most common errors in the Arduino development environment. The following are a few examples of simple statements:

```
delay(1000);

digitalWrite(11, HIGH);

int myValue = 0;

myValue = 150 + 150;
```

Compound statements are made up of several lines of code that include other statements. Functions, such as `setup()` and `loop()`, are in effect compound statements. The `if` statement, covered in greater detail later in this book, is another form of a compound statement. A common example of the `if` statement would look like the following:

```
if (pin2 == HIGH) {
  digitalWrite(13, HIGH);
}
```

This statement will check the condition of the variable `pin2` and will perform the following enclosed statements, to turn pin 13 on, if the variable meets the defined condition, in this case if the state of pin 2 is equal to `HIGH`. In this example, we see that compound statements use **curly braces**, { }, to enclose one or more statements within it, creating a continuous block of code. Curly braces must be used in matching pairs and can encompass as many other statements, both simple and compound, as necessary.

While our example works fine, curly braces are only really needed by the structure of multi-line compound statements. If, as in the most recent example, the conditional statement only includes one other simple statement, the complex statement could be rewritten as a simple statement by removing the curly braces, as in the following syntax:

```
if (pin2 == HIGH) digitalWrite(13, HIGH);
```